MINI PROJECT – ASSETFLOW
IT ASSET LIFECYCLE AND EMPLOYEE OFFBOARDING MANAGEMENT SYSTEM

CS2333 – OBJECT ORIENTED PROGRAMMING USING JAVA MINI PROJECT REPORT

Submitted by

SRI NIDHI R – 251001265

SUMITHRA V – 251001274

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled “AssetFlow – IT Asset Lifecycle and Employee Offboarding Management System” is the bonafide work of:

SRI NIDHI R – 251001265

SUMITHRA V – 251001274

who carried out the mini project work under my supervision.

SIGNATURE

_____
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College
Thandalam, Chennai – 602 105

SIGNATURE

_____
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College
Thandalam, Chennai – 602 105

This mini project report is submitted for CS2333-Object Oriented Programming using Java held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

AssetFlow – IT Asset Lifecycle and Employee Offboarding Management System is a Java-based desktop application intended to organise the management of an organisation's IT assets and the employee records associated with them. The system provides an administrator interface for maintaining employee information, recording IT assets, allocating assets, viewing allocation history, processing returns, reviewing defect reports, and maintaining exit-clearance records. An employee-facing portal provides access to personal asset information, allocation and return history, defect reporting, and exit-clearance status.

The application is developed using Java and Object-Oriented Programming principles, with Java Swing used to construct the graphical user interface. JDBC provides communication between the application and a MySQL database. Maven is used for mini project dependency management and build configuration, and MySQL Connector/J is included as the database driver. The supplied database schema contains related tables for employees, assets, allocations, returns, defect reports, data handovers, exit clearance, user accounts and activity logs.

The mini project demonstrates a structured approach to IT asset lifecycle record management. It supports the recording of allocation and return events, status updates, role-based login paths for administrators and employees, and a central record of selected administrative actions. The report describes the system design and implementation based on the supplied Java source code and SQL schema. Where a database structure exists without a corresponding complete user-interface workflow in the supplied source, that distinction is stated rather than assumed.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
—	Abstract	3
1	INTRODUCTION	5
1.1	Problem Statement	5
1.2	Mini Project Overview	5
1.3	Objectives	5
1.4	Scope	5
2	FUNCTIONALITIES PROVIDED	6
2.1	Functional Requirements	6
2.2	Non-Functional Considerations	6
2.3	Software Requirements	6
3	SYSTEM DESIGN AND FLOW	8
3.1	System Architecture	8
3.2	System Flow	8
3.3	Data Flow Diagram	9
3.4	Use Case Description	10
3.5	Entity Relationship Description	11
4	MODULE DESCRIPTION	13
4.1	Employee Management	13
4.2	Asset Inventory Management	13
4.3	Asset Allocation and History	14
4.4	Asset Return Management	15
4.5	Employee Portal	15
4.6	Defect Reporting and Tracking	16
4.7	Data Handover Tracking	17
4.8	Employee Exit Clearance	17
4.9	Activity Logs and Records	18
4.10	Reports and Records	18

5	IMPLEMENTATION	20
5.1	Technology Stack	20
5.2	OOP Concepts	20
5.3	Database Design	21
5.4	JDBC Connectivity	22
5.5	Application Workflow	22
5.6	Security and Validation	22
6	TESTING AND RESULTS	23
6.1	Testing Approach	23
6.3	Limitations	23
A	Selected Java Source Code Listings	25
7	CONCLUSION AND FUTURE ENHANCEMENTS	28
8	REFERENCES	29

1 – INTRODUCTION

1.1 PROBLEM STATEMENT

Organisations use laptops, monitors, mobile devices, peripherals and other IT equipment to support daily employee activities. When employee details, asset inventory, allocations, returns and exit-clearance information are maintained in separate spreadsheets or paper records, it can become difficult to identify the current holder of an asset, confirm whether an item has been returned, review its condition, or determine whether an employee's exit requirements have been completed. Inconsistent records may also make it difficult to trace previous allocation and return events.

AssetFlow addresses this record-management problem by providing a desktop application connected to a central MySQL database. It brings employee and asset information into a common application and records key lifecycle activities such as allocation, return, defect reporting and exit clearance.

1.2 MINI PROJECT OVERVIEW

AssetFlow is implemented as a Java Swing desktop application. It has separate administrator and employee login paths. The administrator interface provides dashboard information and pages for employee records, asset inventory, allocations, returns, exit clearance, defect reports and activity logs. The employee portal provides an employee-specific view of profile information, assigned assets, allocation history, returns, defect reports and exit-clearance information.

The Java application uses repository classes to perform database operations through JDBC. The MySQL schema stores the operational records in related tables. The Maven build configuration includes MySQL Connector/J as a dependency. The source package also contains model classes representing people, employees, assets and selected operations.

1.3 OBJECTIVES

- Maintain employee information in a structured database.
- Maintain an inventory of IT assets with asset codes, serial numbers, types, conditions and statuses.
- Record asset allocation to employees and retain allocation history.
- Record returned assets, their condition and relevant remarks.
- Allow employees to view their own asset-related records through a portal.
- Provide a mechanism for employees to submit asset defect reports and for administrators to review their status.
- Maintain exit-clearance records covering asset return, data handover and access revocation.
- Record selected administrative actions and display useful operational counts and lists.

1.4 SCOPE

The mini project scope covers desktop-based record management for employees and IT assets. The supplied implementation includes administrator and employee interfaces, login and employee account registration, employee and asset record operations, allocation and return workflows, defect-report views, exit-clearance records and activity logs. The database also contains a data_handovers table. However, the supplied Java source does not show a separate data-handover repository or dedicated data-handover screen; therefore, this report describes that table and the data-handover indicator used by exit clearance without claiming a fully independent handover workflow.

The mini project is not described as a cloud-hosted or browser-based application. No claims are made about integration with enterprise directory services, automated email notifications, barcode scanning, mobile applications, or external IT service-management platforms.

2 – FUNCTIONALITIES PROVIDED

2.1 FUNCTIONAL REQUIREMENTS

- Administrator login and employee login through a role selection on the login interface.
- Employee account registration using employee code, registered email and password confirmation; the source validates required fields, matching passwords and a minimum password length of eight characters.
- Employee record creation and listing through the administrator interface.
- Asset inventory listing and asset creation through the administrator interface.
- Asset allocation to an employee, including a check that the asset is available before allocation.
- Allocation history listing with asset, employee, allocation date, return date and allocation status.
- Asset return recording with actual return date, return condition and remarks.
- Employee portal views for profile, current assets, allocation history, defect reports, returns and exit clearance.
- Employee defect-report submission and administrator defect-report review.
- Exit-clearance record creation and listing with assets-returned, data-handover and access-revoked indicators.
- Activity log recording and retrieval.
- Dashboard counts for employees, assets, available assets and allocated assets.

2.2 NON-FUNCTIONAL CONSIDERATIONS

The supplied source indicates a desktop interface built with Swing and a relational database accessed through JDBC. Repository operations use PreparedStatement and try-with-resources in the inspected classes. Allocation and return workflows use database transactions so that related record changes can be committed together or rolled back when an error occurs. Passwords are processed through a PBKDF2WithHmacSHA256-based utility with a random salt in the supplied PasswordUtil class.

Usability is supported through separate administrator and employee views, tables, forms, labels and dialog messages. Maintainability is supported by separating model, repository and UI packages. The source does not provide measured performance, availability, scalability or usability-test statistics; such values are therefore not reported.

2.3 SOFTWARE REQUIREMENTS

Component	Technology / Version evidence	Purpose
Programming language	Java	Application logic and object-oriented model.
GUI	Java Swing	Desktop windows, forms, navigation and tables.
Database	MySQL	Persistent storage for application records.
Connectivity	JDBC	Communication between Java and MySQL.
Database driver	MySQL Connector/J 9.5.0 (pom.xml)	JDBC driver dependency.
Build tool	Apache Maven	Dependency and mini project build configuration.
IDE	Visual Studio Code	Development environment specified for the mini project.
Database client	MySQL 8.0.46 in supplied dump header	Database schema and SQL management.

3 – SYSTEM DESIGN AND FLOW

3.1 SYSTEM ARCHITECTURE

AssetFlow follows a layered desktop application structure. Swing user-interface classes accept administrator or employee actions. Application and model logic represent employees, assets and operations. Repository classes encapsulate SQL queries and updates. JDBC creates database connections and executes prepared SQL statements against the MySQL database.

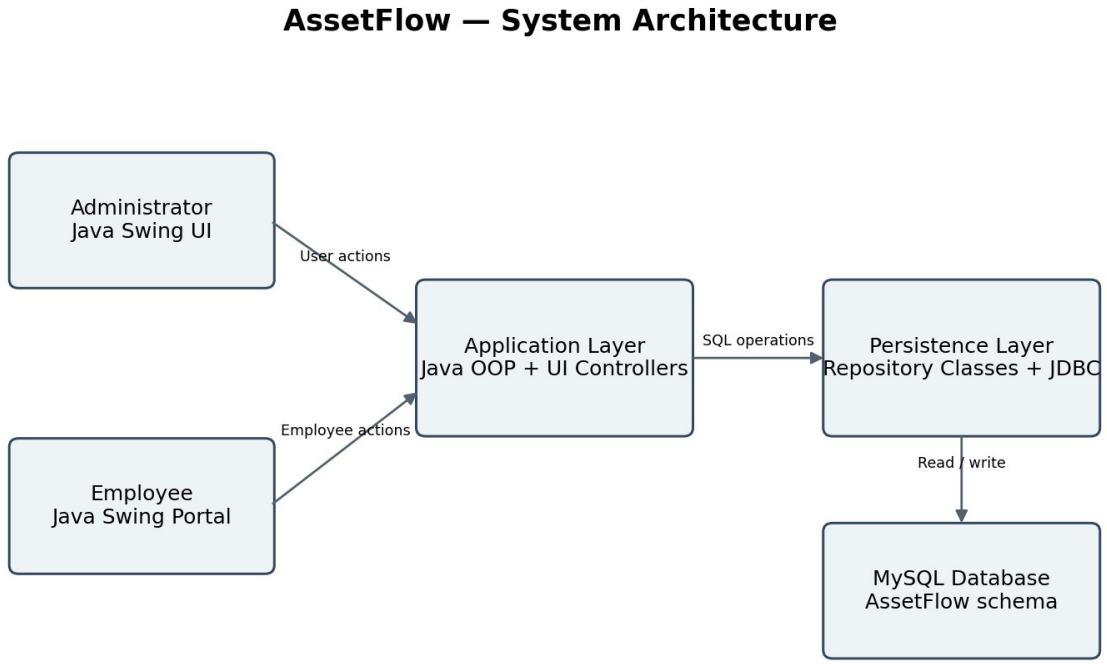


Figure 3.1 — High-level AssetFlow system architecture

3.2 SYSTEM FLOW

1. Start the Java application and display the login window.
2. Select the account role and enter the username and password.
3. Authenticate the account against the user_accounts table.
4. Open the administrator dashboard or the employee dashboard according to the authenticated role.
5. Perform the selected operation through the relevant form or table.
6. Use repository methods and JDBC to read or update MySQL records.
7. Display the resulting records or status message in the interface.
8. Close the application or log out when the session is complete.

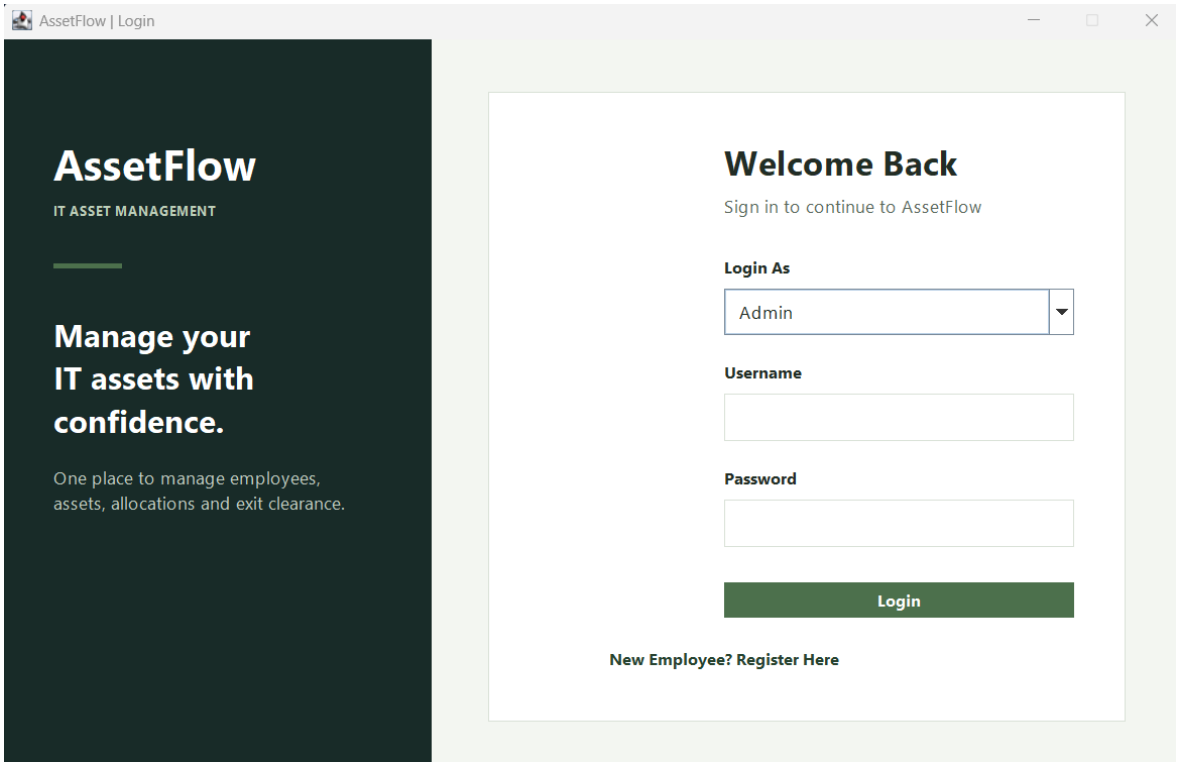


Figure 3.2 — AssetFlow login screen

3.3 DATA FLOW DIAGRAM

The context diagram treats AssetFlow as one system and shows the two primary external actors. Administrators submit employee, asset, allocation, return and clearance operations. Employees use the portal to view personal information and submit defect reports. The application reads and writes the corresponding MySQL records and returns lists, statuses and dashboard information.

AssetFlow — Context Data Flow Diagram



Figure 3.3 — Context-level data flow diagram

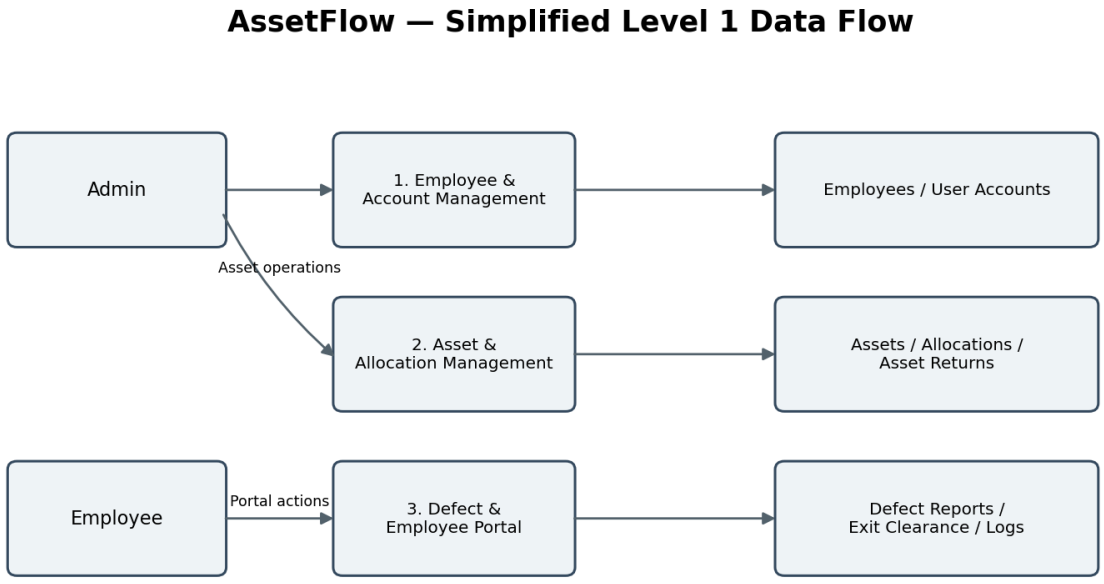


Figure 3.4 — Simplified Level 1 data flow diagram

3.4 USE CASE DESCRIPTION

Actor	Use case	Description
Administrator	Manage employees	Add employee details and view employee records.
Administrator	Manage assets	Add IT assets and view inventory records.
Administrator	Allocate assets	Assign an available asset to an employee and view allocation history.
Administrator	Record returns	Record the return condition and remarks for an allocation.
Administrator	Review defects	View defect reports and their current status.
Administrator	Manage exit clearance	Record asset return, data handover and access-revocation indicators.
Administrator	View activity logs	Review recorded application actions.
Employee	Access portal	View own profile and asset-related records.
Employee	Report defect	Submit a defect report for an associated asset.
Employee	View exit status	View the employee's clearance information.

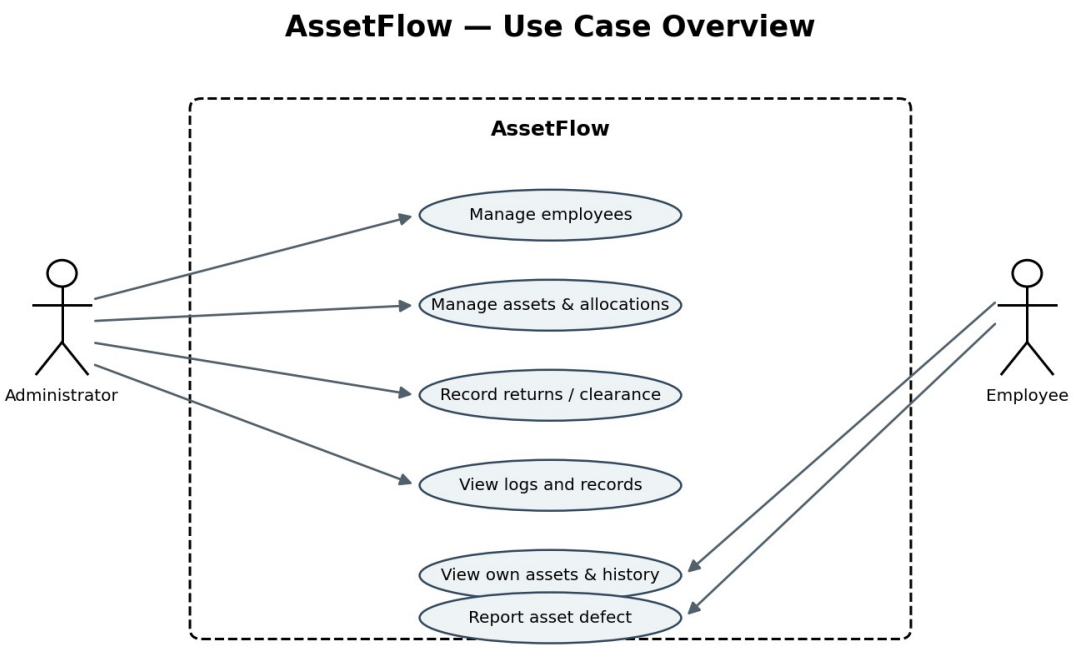


Figure 3.5 — Main use cases supported by the supplied application

3.5 ENTITY RELATIONSHIP DESCRIPTION

The ER overview is based on the CREATE TABLE statements and foreign-key declarations in the supplied assetflowit.sql dump. Employees and assets are parent records for allocations. An allocation refers to one employee and one asset. Asset returns refer to an allocation. Defect reports refer to both an employee and an asset. Data handovers and exit-clearance records refer to employees. User accounts can be associated with an employee; the supplied schema declares employee_id unique in user_accounts and includes a role-related CHECK condition. Activity logs have no declared foreign key in the supplied schema.

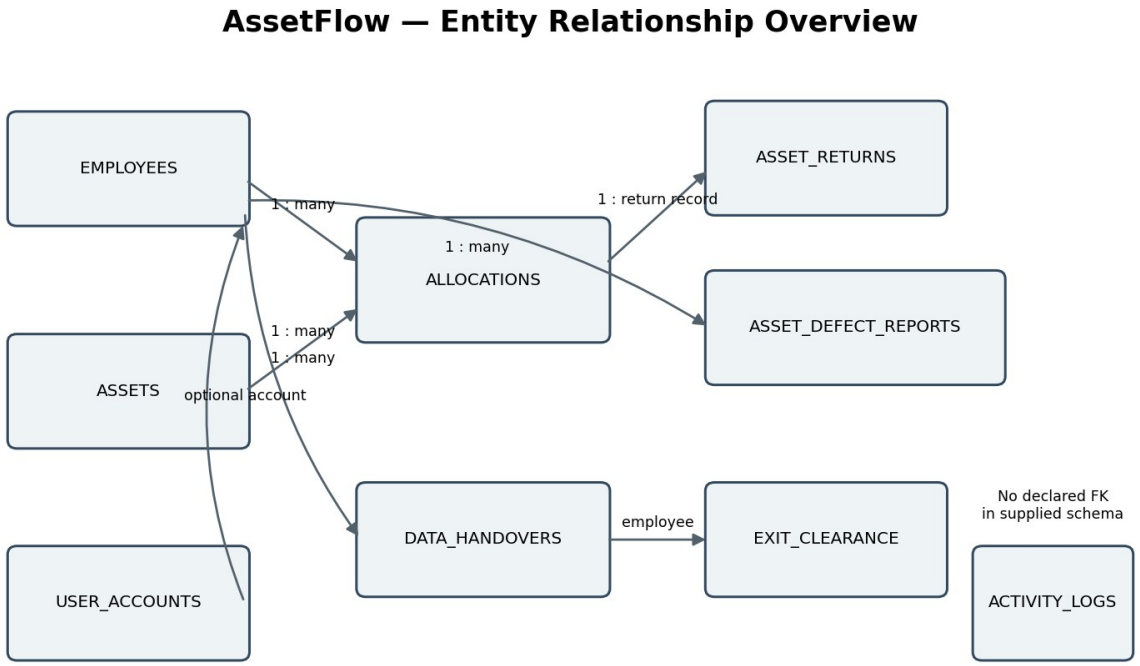


Figure 3.6 — ER relationship overview derived from the supplied SQL schema

The diagram is a relationship overview rather than a complete column-level ERD. The schema permits multiple allocation records for an employee and for an asset over time. The asset_returns table references allocation_id but does not declare it unique in the supplied dump; the operational return method checks allocation status before recording a return.

4 – MODULE DESCRIPTION

4.1 EMPLOYEE MANAGEMENT

Purpose: This module maintains the employee directory used by allocation, defect-reporting and exit-clearance operations. The employees table stores employee code, name, department, designation, email, joining date and employment status. The EmployeeRepository class includes methods to add an employee and retrieve employee records. The administrator interface contains an employee listing and an employee form.

Workflow: The administrator opens the employee page, reviews the available records or enters the required employee details in the form. The repository sends the insert or retrieval request to MySQL through JDBC. Employee identifiers are subsequently used by allocation and other employee-linked records.

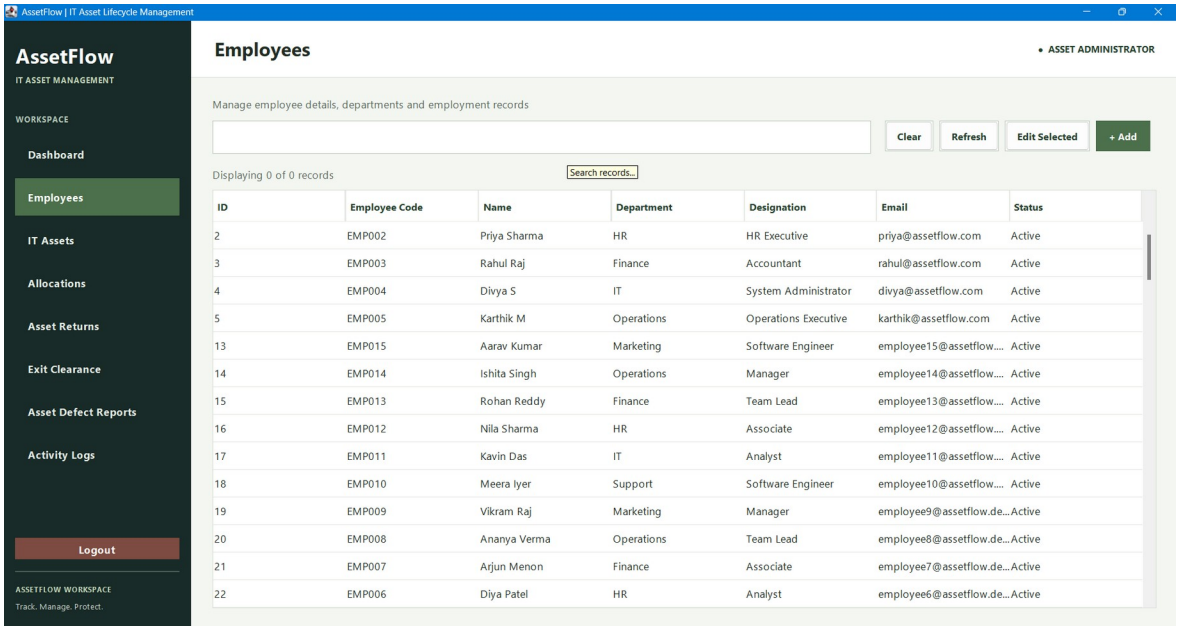


Figure 4.1 — Administrator employee management screen

4.2 ASSET INVENTORY MANAGEMENT

Purpose: This module maintains the organisation's IT asset inventory. The assets table stores an asset identifier, asset code, asset name, asset type, serial number, purchase date, condition and status. The schema declares asset_code and serial_number as unique keys. The AssetRepository class retrieves asset records, while the administrator interface provides an asset listing and asset-entry form.

Workflow: An administrator enters the asset details and saves the record. The asset is stored in MySQL and can later be selected for allocation. The status field distinguishes assets such as Available, Allocated or Maintenance, as reflected in the supplied SQL data and return logic.

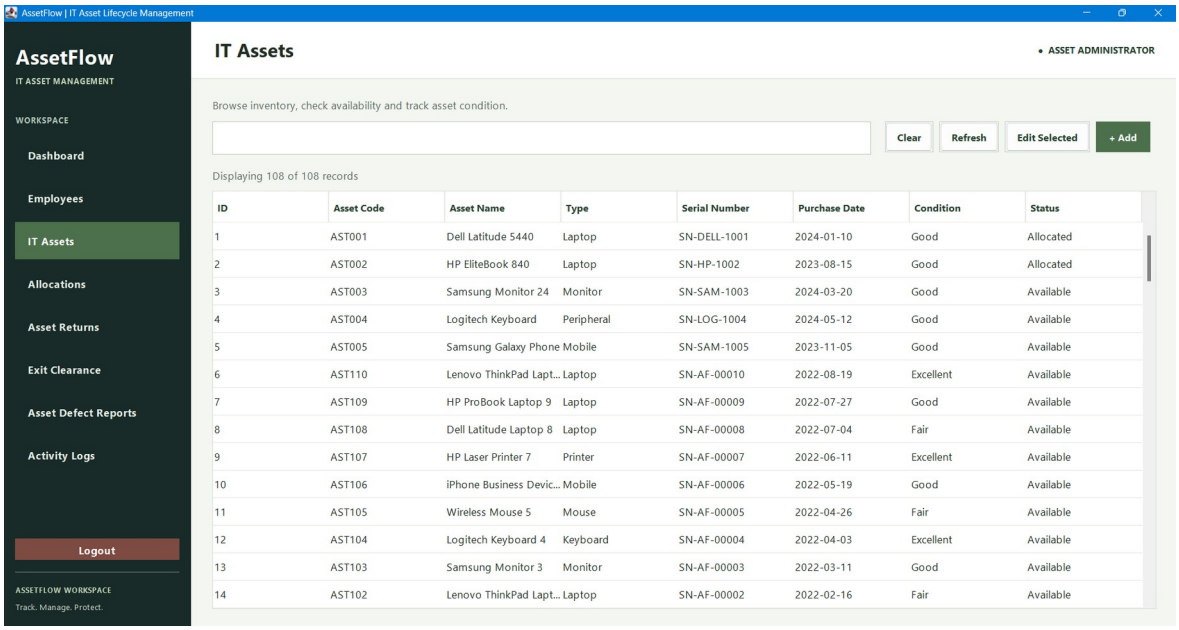


Figure 4.2 — Asset inventory screen

4.3 ASSET ALLOCATION AND HISTORY

Purpose: This module records the assignment of an IT asset to an employee and retains the allocation history. The allocations table links asset_id and employee_id and records allocation_date, return_date and allocation_status. AllocationRepository retrieves joined records showing the asset and employee along with the allocation dates and status.

Workflow: When an administrator allocates an asset, the repository checks its current status using a database query with row locking. If the asset is not available, the operation is rejected through AssetNotAvailableException. If it is available, the application inserts an allocation record and updates the asset status to Allocated within a transaction. The transaction is committed only after the related changes succeed.

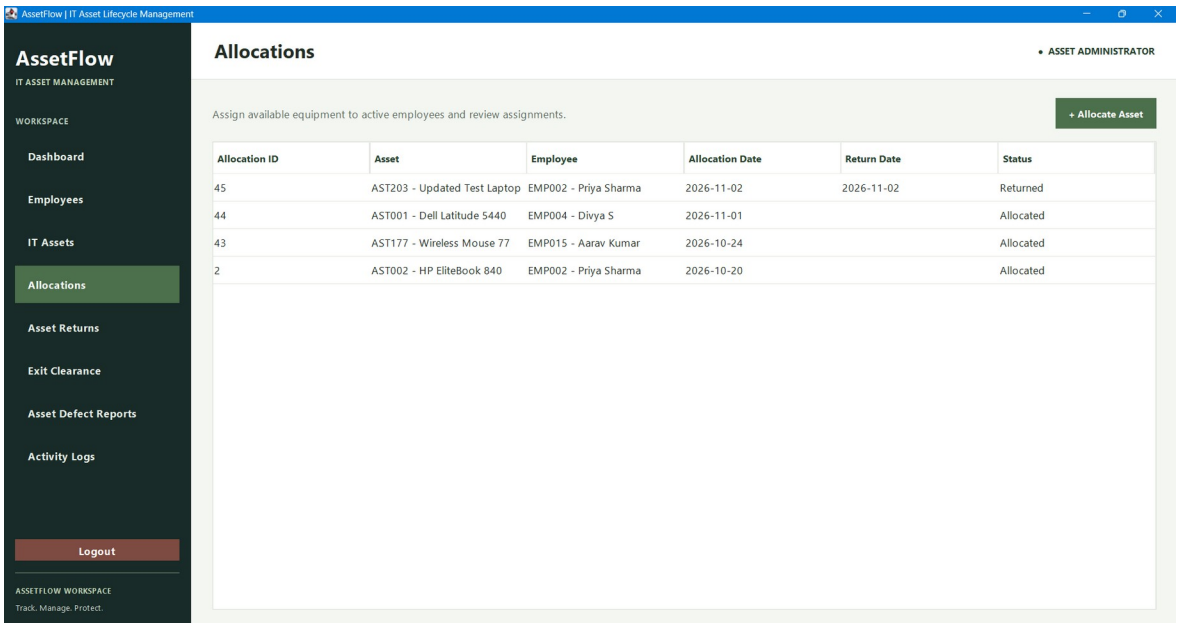


Figure 4.3 — Asset allocation form and allocation history

4.4 ASSET RETURN MANAGEMENT

Purpose: This module records the return of an allocated asset and its condition at the time of return. The asset_returns table stores the related allocation identifier, actual return date, return condition and remarks. AssetReturnRepository implements the return operation and retrieves joined return records.

Workflow: The repository first verifies that the selected allocation exists and has Allocated status. It then inserts the return record, updates the allocation status to Returned and records the return date. The asset status is set to Maintenance when the return condition is Damaged; otherwise it is set to Available. These changes are performed within a transaction with rollback on SQL failure.

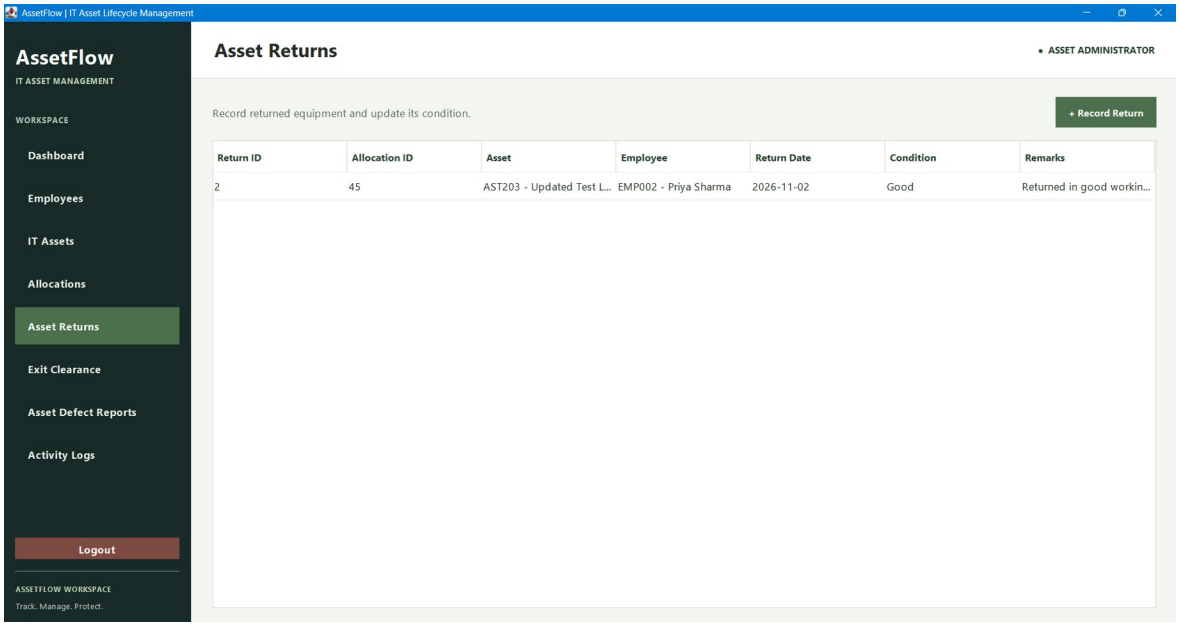


Figure 4.4 — Asset return screen

4.5 EMPLOYEE PORTAL

Purpose: The employee portal provides an employee-specific interface after successful employee authentication. EmployeeDashboardFrame contains pages for dashboard information, profile, assigned assets, allocation history, defect reports, returns and exit clearance. The employee identifier obtained during authentication is used to retrieve the corresponding records.

Workflow: An employee selects the Employee role on the login screen and enters the registered credentials. After successful authentication, the application opens the employee dashboard. The employee can navigate between the available pages and access records associated with their employee ID.

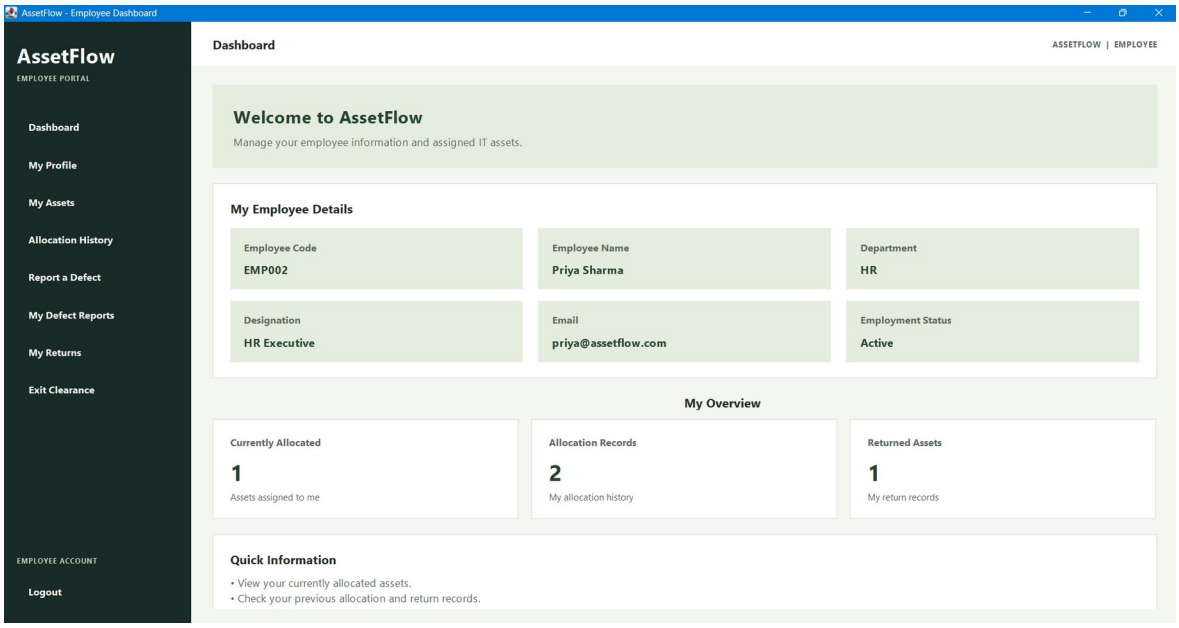


Figure 4.5 — Employee portal dashboard

4.6 DEFECT REPORTING AND TRACKING

Purpose: This module captures problems reported against IT assets. The asset_defect_reports table links each report to an employee and an asset and stores the reported date, description, severity, report status, administrator notes and optional resolution date. The employee dashboard includes a defect-report view and a defect submission form. The administrator interface includes a defect-report listing.

Workflow: An employee selects an associated asset and submits a defect description and severity. The report is stored in the defect-report table. Administrators can review the report and its status. The supplied SQL dump contains an example record whose status is IN PROGRESS; this is database sample data, not a claim about a measured service outcome.

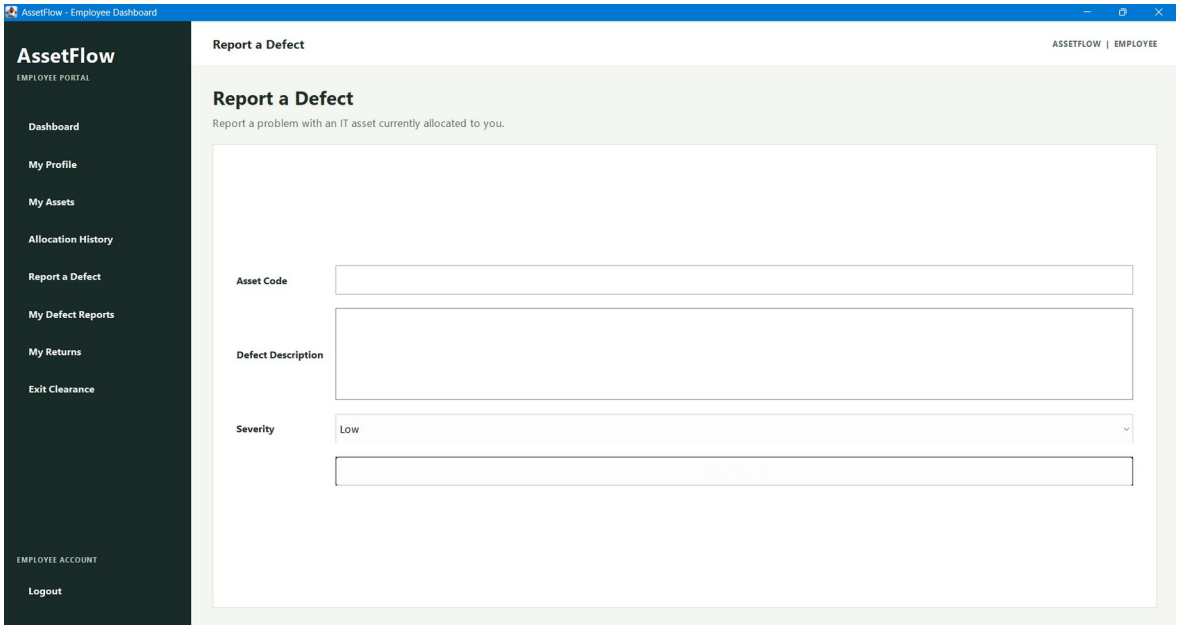


Figure 4.6 — Employee defect reporting screen

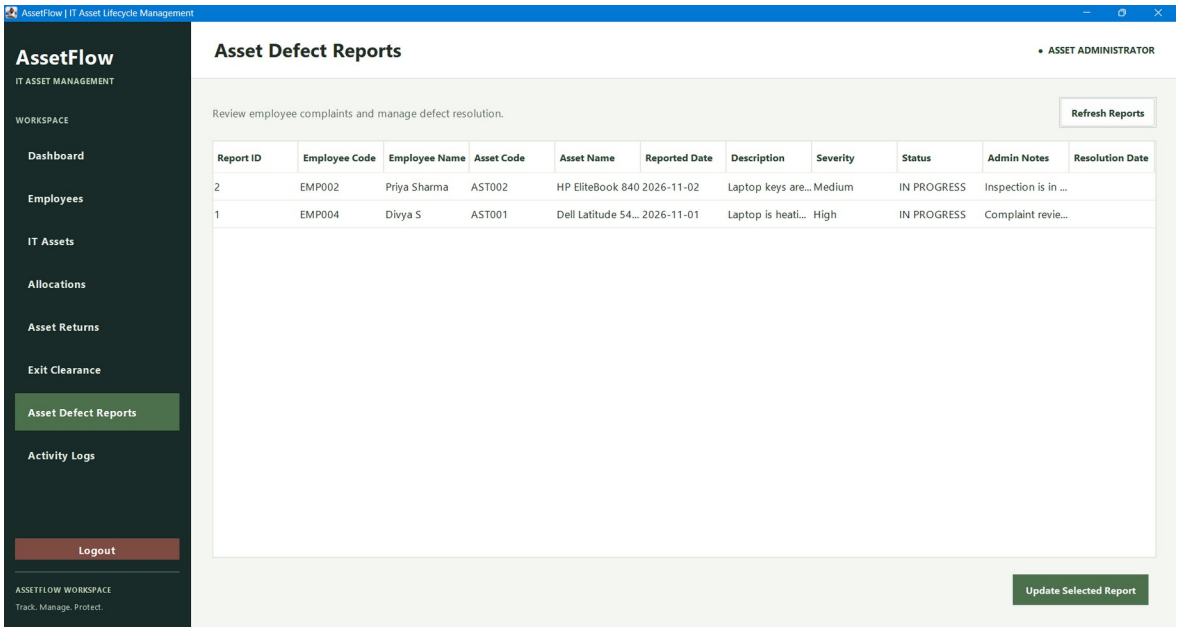


Figure 4.7 — Administrator defect tracking screen

4.7 DATA HANDOVER TRACKING

Purpose: Data handover is an important part of employee exit procedures because work-related information and responsibilities may need to be transferred before an employee leaves. The supplied database includes a data_handovers table with an employee reference, handover date, handover details and handover status.

Implementation boundary: The supplied Java source does not include a dedicated DataHandoverRepository or a separate data-handover screen. The exit-clearance repository does store and retrieve a data_handover Boolean indicator as part of the clearance checklist. Accordingly, the report identifies the data_handovers table as part of the database design but does not claim that a full independent handover workflow is implemented in the supplied UI.

4.8 EMPLOYEE EXIT CLEARANCE

Purpose: This module records whether key exit requirements have been completed for an employee. The exit_clearance table includes employee_id, exit_date, assets_returned, data_handover, access_revoked and clearance_status. ExitClearanceRepository retrieves clearance records joined with employee code and name and saves new clearance records.

Workflow: The administrator selects an employee and records the exit date and the three checklist indicators. The repository sets the clearance status to Completed only when assets_returned, data_handover and access_revoked are all true. Otherwise, the status is Pending. This rule is explicitly implemented in saveClearance.

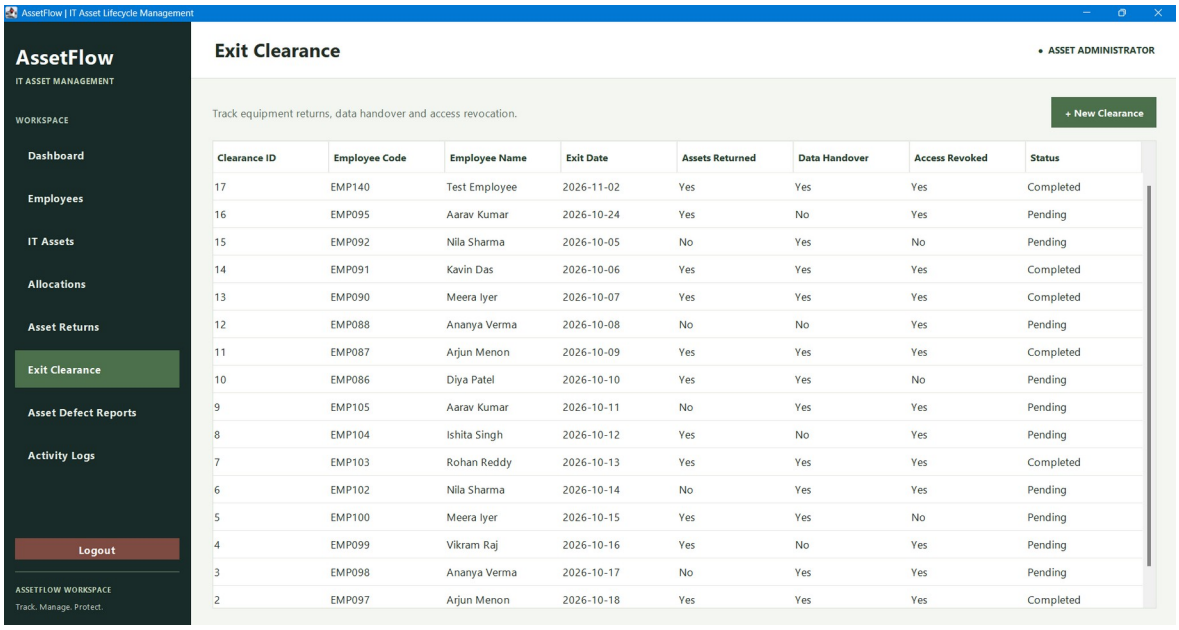


Figure 4.9 — Employee exit clearance screen

4.9 ACTIVITY LOGS AND RECORDS

Purpose: Activity logs provide a chronological record of selected actions. The activity_logs table stores a log identifier, action description and timestamp. ActivityLogRepository includes methods to insert an action description and retrieve the stored logs. The administrator interface contains an activity-log page.

The supplied SQL dump includes sample log descriptions such as employee additions, asset inventory updates, allocation events, defect-report status updates and clearance-related actions. These entries demonstrate the kind of information represented by the log table.

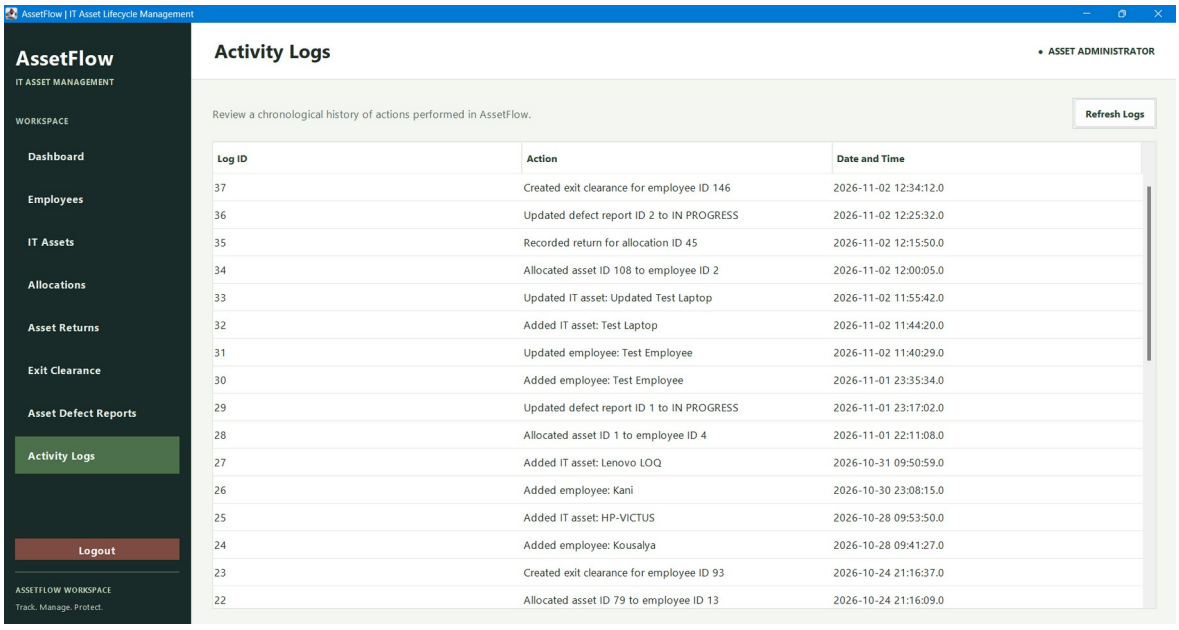


Figure 4.10 — Activity logs screen

4.10 REPORTS AND RECORDS

The dashboard repository provides counts for total employees, total assets, available assets and allocated assets. The administrator UI includes searchable table views for employee, asset, allocation, return,

clearance, defect-report and activity-log records. These views support review of stored information. The supplied source does not demonstrate a separate PDF/Excel report-export function or scheduled reporting facility, so those capabilities are not claimed.

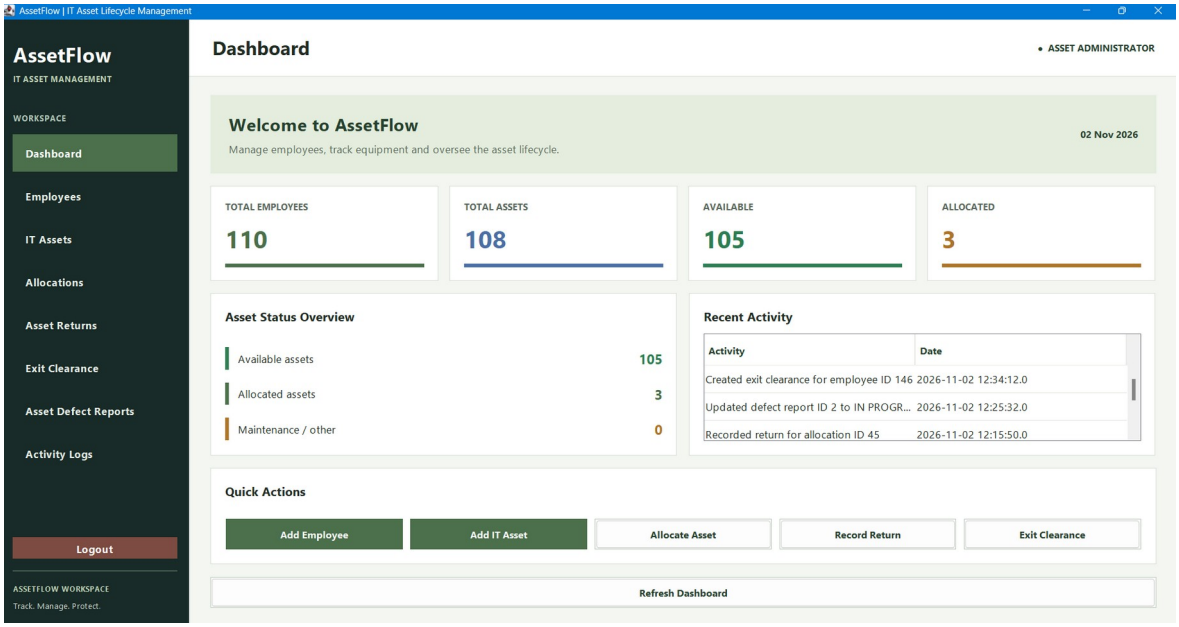


Figure 4.11 — Administrator dashboard and summary cards

5 – IMPLEMENTATION

5.1 TECHNOLOGY STACK

Technology	Role in AssetFlow
Java	Implements application logic, model objects, repository operations and UI event handling.
Java Swing	Builds the desktop interface using JFrame, JPanel, labels, buttons, forms, tables and dialog boxes.
MySQL	Stores employee, asset, allocation, return, defect, handover, clearance, account and activity-log records.
JDBC	Provides connections, prepared SQL statements, result sets and transaction control.
Maven	Defines mini project coordinates and dependencies in pom.xml.
MySQL Connector/J	Provides the JDBC driver for connecting Java to MySQL.
Visual Studio Code	Development environment identified for the mini project.

5.2 OBJECT-ORIENTED PROGRAMMING CONCEPTS

Classes and Objects

The mini project is organised into Java classes that represent application entities and responsibilities. Examples include Employee, Asset, Person, repository classes and Swing window classes. Objects created from these classes hold state and provide operations used by the application.

Encapsulation

Encapsulation is reflected in model classes that keep fields private and expose selected values through getter methods. For example, the Employee and Asset model classes provide accessor methods for their data. Repository classes also group database operations behind methods, reducing the need for UI classes to construct every SQL operation directly.

Inheritance

The Person class is a base class and Employee extends Person in the supplied model package. This represents shared person-related information in a parent class and employee-specific information in a derived class. Swing window classes also extend JFrame to create desktop windows.

Abstraction

Abstraction is used by exposing higher-level repository methods such as addEmployee, getAllAssets, allocateAsset and returnAsset. The caller uses these operations without needing to manage every SQL statement and connection detail. The AssetOperation type is also present as an abstract class in the model package.

Polymorphism

Polymorphism is supported by Java's method overriding mechanism where inherited methods or interface/abstract-type operations can be implemented by specialised classes. The supplied code also contains Swing event-listener implementations and overridden methods such as table-model and listener callbacks. This report does not attribute polymorphism to a specific business operation unless the source explicitly demonstrates it.

5.3 DATABASE DESIGN

The supplied SQL dump identifies the database as assetflow and was generated from MySQL 8.0.46. The following table descriptions are derived from its CREATE TABLE statements. Column names and declared relationships are reproduced from the schema.

Table	Purpose and principal attributes
employees	Employee master: employee_id, employee_code, employee_name, department, designation, email, joining_date, employment_status.
assets	Asset master: asset_id, asset_code, asset_name, asset_type, serial_number, purchase_date, asset_condition, asset_status.
allocations	Allocation link: allocation_id, asset_id, employee_id, allocation_date, return_date, allocation_status.
asset_returns	Return event: return_id, allocation_id, actual_return_date, return_condition, remarks.
asset_defect_reports	Defect records: report_id, employee_id, asset_id, reported_date, defect_description, severity, report_status, admin_notes, resolution_date.
data_handovers	Handover record: handover_id, employee_id, handover_date, handover_details, handover_status.
exit_clearance	Exit checklist: clearance_id, employee_id, exit_date, assets_returned, data_handover, access_revoked, clearance_status.
user_accounts	Login accounts: account_id, username, password_hash, account_role, employee_id, account_status, created_at.
activity_logs	Action history: log_id, action_description, action_date.

5.3.1 Relationships

- employees (1) to allocations (many): allocations.employee_id references employees.employee_id.
- assets (1) to allocations (many): allocations.asset_id references assets.asset_id.
- allocations to asset_returns: asset_returns.allocation_id references allocations.allocation_id.
- employees (1) to asset_defect_reports (many), and assets (1) to asset_defect_reports (many).
- employees (1) to data_handovers (many) and employees (1) to exit_clearance (many), as declared by the foreign keys.
- employees to user_accounts: user_accounts.employee_id references employees.employee_id; employee_id is unique in user_accounts. The schema CHECK condition requires an employee reference for EMPLOYEE accounts and a null employee reference for ADMIN accounts.
- activity_logs has no declared foreign key in the supplied schema.

5.3.2 Example SQL Statements

The following examples illustrate queries that match the supplied schema. They are documentation examples and do not include connection credentials.

```
-- List assets with their current status
SELECT asset_id, asset_code, asset_name, asset_type, asset_status
FROM assets;

-- View allocation history with employee and asset details
SELECT al.allocation_id, e.employee_code, e.employee_name,
    a.asset_code, a.asset_name, al.allocation_date,
    al.return_date, al.allocation_status
FROM allocations al
JOIN employees e ON e.employee_id = al.employee_id
JOIN assets a ON a.asset_id = al.asset_id
ORDER BY al.allocation_id DESC;

-- Review employee exit-clearance records
SELECT e.employee_code, e.employee_name, ec.exit_date,
    ec.assets_returned, ec.data_handover,
    ec.access_revoked, ec.clearance_status
FROM exit_clearance ec
JOIN employees e ON e.employee_id = ec.employee_id;
```

5.4 JDBC CONNECTIVITY

DatabaseConnection is the central connection utility in the Java source. It obtains a JDBC Connection using DriverManager and the configured database URL and credentials. Repository classes call DatabaseConnection.getConnection() to communicate with MySQL. The inspected repositories use PreparedStatement for parameterised queries and ResultSet to process retrieved rows. Connection, statement and result-set resources are generally managed with try-with-resources.

The supplied mini project includes MySQL Connector/J 9.5.0 in pom.xml. The actual database password is intentionally not reproduced in this report. Deployment configuration should keep credentials private and should use a local configuration appropriate to the machine on which the application runs.

5.5 APPLICATION WORKFLOW

The application entry point launches the desktop interface. The login screen asks the user to choose Admin or Employee and enter credentials. UserAccountRepository verifies the account role and password hash. An administrator is directed to AssetFlowUI, while an employee is directed to EmployeeDashboardFrame with the authenticated employee identifier. From the selected page, Swing event handlers call repository methods and display retrieved data in tables or forms.

5.6 SECURITY AND VALIDATION

The supplied account-registration code checks that required fields are present, that the password and confirmation match, and that the password contains at least eight characters. PasswordUtil generates a random salt and derives a PBKDF2WithHmacSHA256 hash. Authentication verifies the entered password against the stored hash and checks the account role and ACTIVE status. The SQL schema also defines account roles and an account-status field.

Allocation logic checks asset availability before creating an allocation. Return logic verifies that the allocation is still marked Allocated. Both operations use transaction handling to keep related database updates consistent. These are implementation observations from the supplied source; no independent penetration test or formal security audit was supplied.

6 – TESTING AND RESULTS

6.1 TESTING APPROACH

The supplied mini project archive includes a JUnit 3.8.1 test dependency and a basic AppTest class containing a single assertion that the application test method returns true. This is a placeholder-style unit test and does not establish comprehensive functional coverage. The source and SQL dump were inspected to document the implemented workflows. The following table is a proposed manual verification checklist; it is not presented as a record of tests already executed.

Test area	Suggested verification procedure	Expected observation (not a recorded result)
Login role	Try an active administrator and employee account with the matching role.	The correct role-specific interface should open.
Employee registration	Enter a valid active employee code, registered email and matching password of at least eight characters.	Account creation should be accepted if the employee record is eligible and no account exists.
Asset allocation	Allocate an asset whose status is Available.	Allocation record should be inserted and asset status changed to Allocated.
Unavailable asset	Attempt allocation when asset status is not Available.	Allocation should be rejected by the availability check.
Asset return	Return an allocation with a return condition.	Return record, allocation status and asset status should be updated together.
Defect reporting	Submit a defect report for an employee and asset.	A report should be stored and appear in the appropriate listing.
Exit clearance	Save clearance with all three indicators true, then with one indicator false.	Status should be Completed for all true; otherwise Pending.
Activity logs	Perform a logged action and open the log page.	The action description and timestamp should be retrievable.

6.3 LIMITATIONS

- The supplied Java source does not show a dedicated data-handover repository or standalone data-handover UI, although the database table exists and exit clearance stores a data-handover indicator.
- The supplied source does not demonstrate export of reports to PDF or spreadsheet formats.
- The included JUnit test class contains only a basic assertion and does not verify the business workflows.
- The SQL dump contains sample/demo records; these should not be interpreted as production organisational data.
- The mini project is a desktop application; browser-based or mobile access is not demonstrated in the supplied source.

APPENDIX A — SELECTED JAVA SOURCE CODE LISTINGS

The following listings show only the principal logic used in the application. They are concise extracts from the supplied Java source files; supporting imports, variable declarations, exception handling and surrounding class code are omitted where they are not necessary to understand the main operation.

A.1 Application Entry Point — App.java

Starts the Swing application on the Event Dispatch Thread.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new LoginFrame());
}
```

A.2 JDBC Connection — DatabaseConnection.java

Creates a JDBC connection to the AssetFlow MySQL database. Database credentials are intentionally not printed in this report.

```
public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(URL, USER, PASSWORD);
}
```

A.3 Employee Inheritance — Employee.java

Employee inherits the common name and email properties from Person and initializes its employee-specific fields.

```
public class Employee extends Person {
    private int employeeId;
    private String employeeCode;
    private String department;
    private String designation;
    private String employmentStatus;

    public Employee(int employeeId, String employeeCode,
            String employeeName, String department,
            String designation, String email, String employmentStatus) {
        super(employeeName, email);
        this.employeeId = employeeId;
        this.employeeCode = employeeCode;
        this.department = department;
        this.designation = designation;
        this.employmentStatus = employmentStatus;
    }
}
```

A.4 Employee Retrieval — EmployeeRepository.java

Uses PreparedStatement and ResultSet to retrieve employee details and map each row to an Employee object.

```
String sql = "SELECT employee_id, employee_code, employee_name, " +
    "department, designation, email, employment_status " +
    "FROM employees ORDER BY employee_id";

try (Connection con = DatabaseConnection.getConnection();
     PreparedStatement ps = con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        Employee employee = new Employee(
            rs.getInt("employee_id"),
            rs.getString("employee_code"),
            rs.getString("employee_name"),
            rs.getString("department"),
            rs.getString("designation"),
            rs.getString("email"),
            rs.getString("employment_status"));
        employees.add(employee);
    }
}
```

A.5 Asset Allocation — AllocationRepository.java

The main allocation logic checks that the asset is available, inserts the allocation, updates the asset status and commits the transaction. The actual method also rolls back on failure.

```
connection.setAutoCommit(false);

if (!"Available".equalsIgnoreCase(status)) {
    throw new AssetNotAvailableException(
        "Asset cannot be allocated because it is not available.");
}

insert.setInt(1, assetId);
insert.setInt(2, employeeId);
insert.executeUpdate();

update.setInt(1, assetId);
update.executeUpdate();

connection.commit();
```

A.6 Asset Return — AssetReturnRepository.java

Records the return and updates both allocation and asset status. Damaged assets are marked for maintenance.

```
String newAssetStatus =
   "Damaged".equalsIgnoreCase(returnCondition)
      ? "Maintenance" : "Available";

returnStatement.setInt(1, allocationId);
returnStatement.setDate(2, today);
returnStatement.setString(3, returnCondition);
returnStatement.setString(4, remarks);
returnStatement.executeUpdate();

allocationUpdate.setDate(1, today);
allocationUpdate.setInt(2, allocationId);
allocationUpdate.executeUpdate();

assetUpdate.setString(1, newAssetStatus);
assetUpdate.setInt(2, assetId);
assetUpdate.executeUpdate();
```

A.7 Exit Clearance — ExitClearanceRepository.java

Clearance is marked Completed only when asset return, data handover and access revocation are all confirmed.

```
String status;
if (assetsReturned && dataHandover && accessRevoked) {
   status = "Completed";
} else {
   status = "Pending";
}
```

A.8 Activity Logging — ActivityLogRepository.java

Stores a description of an application action using a parameterized SQL statement.

```
String sql =
   "INSERT INTO activity_logs (action_description) VALUES (?)";

try (Connection con = DatabaseConnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql)) {
   ps.setString(1, description);
   return ps.executeUpdate() > 0;
}
```

A.9 User Authentication — UserAccountRepository.java

Checks the account role and verifies the entered password against the stored password hash.

```
if (!storedRole.equals(role)) {
   return null;
}

if (!PasswordUtil.verifyPassword(password, storedHash)) {
   return null;
}

return new AuthenticatedUser(storedRole, employeeId);
```

7 – CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

AssetFlow is a Java Swing desktop application designed to organize IT asset lifecycle activities and employee offboarding records. The supplied implementation connects employee and asset information with allocation and return operations, provides administrator and employee interfaces, supports defect-reporting views, maintains exit-clearance indicators and records selected activity descriptions. MySQL stores the application data, while JDBC enables the Java application to communicate with the database.

The mini project demonstrates the use of Java classes and objects, inheritance, encapsulation, Swing-based desktop interface development, relational database design and transaction-based data updates. The implementation provides a foundation for maintaining traceable asset and employee records. Further development can strengthen data handover tracking, reporting, automated testing and deployment. Thus, AssetFlow addresses the central record-management needs considered in this mini project while leaving clearly identified areas for future enhancement.

7.2 FUTURE ENHANCEMENTS

- Implement a dedicated data-handover screen and repository to create, update and review individual handover records.
- Add exportable reports for asset inventory, allocation history, returns, defects and exit-clearance status.
- Introduce configurable audit details such as the acting account and related record identifier in activity logs, where appropriate.
- Add stronger validation and automated test coverage for repository and UI workflows.
- Provide asset search filters by type, status, condition, employee and purchase date.
- Add notification support for overdue returns, unresolved defects and incomplete exit-clearance items.
- Consider role and permission refinement for sensitive administrative operations.
- Evaluate deployment packaging and backup/restore procedures for the MySQL database.

8 – REFERENCES

- Oracle. Java Platform Documentation. https://docs.oracle.com/en/java/
- Oracle. Java Swing Tutorial. https://docs.oracle.com/javase/tutorial/uiswing/
- MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
- MySQL. MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/
- Apache Maven Project. Maven Documentation. https://maven.apache.org/guides/
- mini project source code supplied for AssetFlow – IT Asset Lifecycle and Employee Offboarding Management System.